

HW1: Mid-term assignment report

Ana Alexandra Antunes [876543], v2025-10-22

1.1 Overview of the work	1
1.2 Current implementation (faults & extras)	2
1.3 Use of generative AI	2
2.1 Functional scope and supported interactions	2
2.2 System implementation architecture	4
2.3 API for developers	7
3.1 Overall strategy for testing	7
3.2 Unit and integration testing	7
3.3 Acceptance testing	9
3.4 Functional testing	10
3.5 Non-functional testing	10
3.6 Code quality analysis	11

1 Introduction

1.1 Overview of the work

Este relatório apresenta o projeto individual intermédio de TQS, cobrindo as funcionalidades do produto e a estratégia de garantia de qualidade adotada.

A aplicação, a que chamo “ZeroMonos”, é uma solução web simples para gestão de pedidos de recolha de resíduos por cidadãos. O objetivo é permitir que qualquer cidadão submeta um pedido indicando município, descrição, data e janela horária, receba um código (token) para consulta posterior e acompanhe o estado do pedido e o cancele, se necessário. O ciclo de vida do pedido é rastreado através de estados (RECEBIDO, ATRIBUIDO, EM_PROG, CONCLUIDO, CANCELADO), preservando o histórico de transições.

A app usa Spring Boot no backend (REST), páginas estáticas HTML/JS no frontend e testes BDD com Cucumber + Selenium para validar o fluxo ponta-a-ponta. A cobertura é analisada com JaCoCo.

1.2 Current implementation (faults & extras)

Os objetivos básicos foram todos concretizados com sucesso.

Foi ainda adicionado:

- Histórico de estados: o domínio inclui BookingStateHistory, registrando todas as transições de estado.
- Tratamento uniforme de erros: GlobalExceptionHandler centraliza mensagens de validação e erros inesperados.

1.3 Use of generative AI

Usei assistentes de IA exclusivamente para gerar:

- As páginas HTML e ficheiros JS do frontend (estrutura e base funcional).
- A base (esqueleto) dos ficheiros Java que não são de testes (por exemplo, classes de controlador, entidades e configuração inicial).
- Geração dos Loggers.

O que preferi não delegar:

- Testes: os cenários Cucumber e todos os testes.
- Regras de negócio e validações: lógica de domínio (ex.: estados, histórico, validações de data/descrição/município) e mapeamento JPA.
- Decisões de design e mensagens: definição da API, mensagens de erro/validação e organização dos pacotes/ficheiros foram decididas por mim, garantindo coerência e clareza.

2 Product specification

2.1 Functional scope and supported interactions

Cidadão

- Cria um pedido de recolha indicando município, descrição, data e janela horária.
- Recebe um token (código) para consultar/cancelar o pedido.
- Consulta estado do pedido pelo token e, se aplicável, cancela.

Staff

- Visualiza pedidos por município.
- Atualiza o estado do pedido: RECEBIDO → EM_PROG → CONCLUIDO.

Não há autenticação/roles neste MVP.

Gestão de Pedidos

Município:

Ex: Lisboa

Carregar Pedidos

Município	Descrição	Data	Horário	Estado	Ações
Abrantes	sofá	2025-11-21	09:00-11:00	RECEBIDO	Em Progresso
Alcoutim	cama de 3 metros	2025-11-26	09:00-11:00	EM_PROG	Concluído

Página <http://localhost:8080/staff.html> - visão do staff

Pedido de Recolha de Lixo

Novo Pedido de Recolha

Município:

Abrantes

Descrição:

Data desejada:

03/11/2025

Horário:

09:00-11:00

Submeter Pedido

Consultar ou Cancelar Pedido

Introduza o seu código de pedido:

Ex: 502be1ad-e9d3-44f1-a

Ver Pedido

Página <http://localhost:8080/index.html> - visão do cidadão

O staff UI usa apenas três transições: RECEBIDO → EM_PROG → CONCLUIDO. A ação de cancelar está exposta na página do cidadão.

Regras de negócio (principais)

- Validações no domínio (Booking.validateSelf):
- Município obrigatório, descrição obrigatória, data obrigatória, timeslot obrigatório.
- Não permite fim de semana; data com ≥ 3 dias de antecedência.
- Validações no serviço (BookingService.createBooking):
- Descrição com pelo menos 3 chars.
- Limite diário por município (5 pedidos/dia).
- Impede conflitos de horário (mesma data + timeslot não cancelado).
- Limite de reservas “ativas” (não canceladas/concluídas) no sistema (MAX_ACTIVE_BOOKINGS = 3).

2.2 System implementation architecture

Arquitetura (camadas)

boundary (REST)

- BookingController: expõe CRUD-like para pedidos e mudanças de estado.
- MunicipioController: expõe lista de municípios.
- GlobalExceptionHandler: normaliza respostas de erro.

service (negócio)

- BookingService: validações de regras de negócio, transições de estado, consultas ao repositório.
- MunicipioService: consulta de municípios à API pública (GeoAPI).

data (persistência)

- Booking (entidade JPA) com @Enumerated(EnumType.STRING) em status, e @OneToMany para stateHistory.
- BookingState, BookingStateHistory.
- BookingRepository (Spring Data JPA).

Tecnologias/frameworks

Backend: Spring Boot (Web, Data JPA, Validation), Hibernate,, HikariCP.

DB: H2 em memória.

Frontend: HTML + water.css, JS vanilla (fetch API), páginas em /static.

Testes e qualidade:

JUnit 5 (Jupiter)

Spring Boot Test:

- @SpringBootTest, @AutoConfigureMockMvc, MockMvc (integração de controllers/HTTP).

- @LocalServerPort para levantar servidor em RANDOM_PORT nos funcionais e performance.

Cucumber 7 + JUnit Platform.

Selenium WebDriver 4.

AssertJ e JUnit Assertions

Java 11+ HttpClient

JaCoCo

Sonar

2.3 API for developers

Base URL: **http://localhost:8080**

GET /api/municipios

Descrição: devolve a lista de municípios (strings)

Response 200:

```
["Lisboa", "Porto", ...]
```

Erros:

- 500 com:

```
{ "error": "Ocorreu um erro inesperado" }
```

POST /api/bookings

Descrição: cria um novo pedido

Request body (JSON):

```
{
  "municipality": "string",    // obrigatório, tem de existir na GeoAPI
  "description": "string",     // ≥ 3 chars
  "requestedDate": "string",   // ISO yyyy-MM-dd, ≥ hoje+3, não
fim-de-semana
  "timeSlot": "string"        // "HH:MM-HH:MM"
}
```

Response 200:

```
{
  "municipality": "Lisboa",
  "description": "Teste funcional",
  "requestedDate": "2025-11-19",
  "timeSlot": "09:00-11:00",
  "token": "502be1ad-...",
}
```

```

    "status": "RECEBIDO",
    "stateHistory": [
      { "timestamp": "...", "status": "RECEBIDO" }
    ]
  }

```

Erros (400):

- “Descrição inválida — demasiado curta”
- “Município inválido: X”
- “Limite de pedidos atingido para este dia”
- “Não é possível reservar dois serviços no mesmo horário.”
- “O cidadão já atingiu o limite de reservas ativas.”

GET /api/bookings/{token}

Descrição: obtém um pedido pelo token

Response 200: Booking (mesmos campos do POST)

Erros:

- 500 com:


```
{ "error": "Ocorreu um erro inesperado" }
```

se o serviço lançar RuntimeException: “Booking não encontrado”

GET /api/bookings?municipality={nome}

Descrição: lista pedidos por município (ou todos se o parâmetro for omitido)

Response 200:

[Booking, Booking, ...]

PUT /api/bookings/{token}?status={RECEBIDO|EM_PROG|CONCLUIDO|CANCELADO}

Descrição: atualiza o estado do pedido (com validação de transições)

Regras de transição atuais:

- RECEBIDO → EM_PROG | CANCELADO
- EM_PROG → CONCLUIDO | CANCELADO
- CONCLUIDO/CANCELADO → (sem transições)

Response 200: Booking atualizado

Erros:

- 400 Estado inválido (quando status não é um valor do enum)
- 400 Transição inválida de X para Y
- 400 Reserva não encontrada para o token fornecido

DELETE /api/bookings/{token}

Descrição: cancela o pedido (define estado CANCELADO)

Response 200:

```
{  
  "status": "CANCELADO",  
  ...  
}
```

3 Quality assurance

3.1 Overall strategy for testing

Foi usada uma estratégia mista (pirâmide de testes):

- Unit tests para fixar regras de domínio (ex.: validações de Booking e evolução de estados).
- Integration tests sobre a API REST (Spring Boot Test/MockMvc) para validar controladores, serialização e handler de erros.
- Acceptance tests BDD com Cucumber + Selenium para validar o fluxo ponta-a-ponta no browser (formulários do cidadão e interações básicas do staff).
- Cobertura com JaCoCo e análise estática com Sonar.

BDD: os cenários descrevem o comportamento esperado; as Step Definitions implementam a automação no Selenium. O runner usa JUnit Platform Suite + cucumber-junit-platform-engine.

Não foi seguido TDD estrito em todo o projeto. Usou-se TDD nas regras de domínio mais críticas e BDD para os casos de aceitação.

3.2 Unit and integration testing

Unit Test

Ficheiro-chave: BookingTest.java

Objetivo: Fixar regras do domínio e contratos da entidade Booking.

O que validam:

- Estado inicial e token: criação começa em RECEBIDO e gera token único.
- Histórico de estados: mantém ordem e regista timestamp.
- Validações de negócio no próprio domínio: campos obrigatórios, data não ao fim de semana e com antecedência mínima.

Tecnologias: JUnit 5 (Jupiter), JUnit Assertions (e alguns asserts de ordem/timestamps).

```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.zeromonos.BookingTest
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.048 s - in com.example.zeromonos.BookingTest
[INFO] Results:
[INFO]
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.13:report (report) @ zeromonos ---
[INFO] Loading execution data file /Users/lucas/Desktop/TQS/tqs-indiv-portfolio-lucasruivo/HW1/target/jacoco.exec
[INFO] Analyzed bundle 'zeromonos' with 9 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.467 s
[INFO] Finished at: 2025-11-03T22:45:58Z
[INFO] -----

```

Service Test (com mocks)

Ficheiro-chave: BookingServiceTest.java

Objetivo: Testar a lógica de orquestração do BookingService isolando repositórios e serviços externos (municípios).

O que validam:

- Criação válida (fluxo feliz) e interações com BookingRepository.
- Rejeições por regras de negócio de serviço:
- Município inválido (mock de MunicipioService).
- Limite diário por município.
- Conflito de horário no mesmo dia.
- Limite de reservas ativas.
- Descrição demasiado curta.
- Transições de estado válidas e bloqueio de transições proibidas.

Tecnologias: JUnit 5, Mockito (mocks/stubs), JUnit Assertions.

```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.zeromonos.BookingServiceTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.479 s -- in com.example.zeromonos.BookingServiceTest
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.13:report (report) @ zeromonos ---
[INFO] Loading execution data file /Users/lucas/Desktop/TQS/tqs-indiv-portfolio-lucasruivo/HW1/target/jacoco.exec
[INFO] Analyzed bundle 'zeromonos' with 9 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.713 s
[INFO] Finished at: 2025-11-03T22:49:19Z
[INFO] -----

```

Integration Tests

Ficheiro-chave: BookingIntegrationTest.java

Objetivo: Validar a API fim-a-fim no servidor Spring (controladores, serialização, handler de erros) com MockMvc.

O que validam:

- POST /api/bookings cria pedido com estado RECEBIDO.
- GET /api/bookings/{token} devolve o pedido.
- PUT estado (ex.: EM_PROG) atualiza corretamente; estado inválido devolve 400 e mensagem via GlobalExceptionHandler.
- GET lista (com e sem filtro de municipality).
- DELETE cancela (estado CANCELADO).
- Rejeições por regras (descrição curta, duplicado no mesmo horário).

Tecnologias: Spring Boot Test, MockMvc, JUnit 5.


```
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.736 s -- in com.example.zeromonos.BookingIntegrationTest
[INFO] Results:
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.13:report (report) @ zeromonos ---
[INFO] Loading execution data file /Users/lucas/Desktop/TQS/tqs-indiv-portfolio-lucasruivo/HW1/target/jacoco.exec
[INFO] Analyzed bundle 'zeromonos' with 9 classes
[INFO] BUILD SUCCESS
[INFO] Total time: 4.021 s
[INFO] Finished at: 2025-11-03T22:50:42Z
[INFO]
```

3.3 Acceptance testing

Ficheiros:

Runner: RunCucumberTest.java

Spring config: CucumberSpringConfiguration.java

Steps: CitizenSteps.java

Feature: citizen_booking.feature

Objetivo: Especificar comportamento do ponto de vista do utilizador (cidadão) e validar a experiência ponta-a-ponta no browser.

O que validam (cenários):

- Happy path: criar pedido → ver token → consultar → cancelar (estado CANCELADO).
- Negativos: município inválido, descrição curta, token inexistente.

Implementação:

- Selenium WebDriver com esperas explícitas (WebDriverWait) e interação robusta com <select> via Select.
- Execução headless opcional (-Dheadless/HEADLESS/CI).
- Spring arrancado em DEFINED_PORT para estabilidade dos testes de UI.

Tecnologias: Cucumber 7 (gherkin + junit-platform-engine + cucumber-spring), Selenium 4, Spring Boot Test, AssertJ/JUnit.

```
4 Scenarios (4 passed)
19 Steps (19 passed)
0m21,008s

Share your Cucumber Report with your team at https://reports.cucumber.io
Activate publishing with one of the following:

src/test/resources/cucumber.properties:  cucumber.publish.enabled=true
src/test/resources/junit-platform.properties:  cucumber.publish.enabled=true
Environment variable:  CUCUMBER_PUBLISH_ENABLED=true
JUnit:  @CucumberOptions(publish = true)

More information at https://cucumber.io/docs/cucumber/environment-variables/

Disable this message with one of the following:

src/test/resources/cucumber.properties:  cucumber.publish.quiet=true
src/test/resources/junit-platform.properties:  cucumber.publish.quiet=true

[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 21.03 s -- in com.example.zeromonos.RunCucumberTest
[INFO] Results:
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
```

3.4 Functional testing

Ficheiros:

CitizenFunctionalTest.java

StaffFunctionalTest.java

Objetivo: Exercitar as páginas reais (index.html e staff.html) com o servidor em RANDOM_PORT.

O que validam:

- Cidadão: criação/consulta/cancelamento no index.html e mensagens de erro esperadas.
- Staff: listagem por município, transições RECEBIDO → EM_PROG → CONCLUIDO, ausência de ações no final.

Tecnologias: Spring Boot Test (@LocalServerPort), WebDriverManager (setup binário), Selenium 4, AssertJ.

```
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.59 s -- in com.example.zeromonos.CitizenFunctionalTest
[INFO] Results:
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.13:report (report) @ zeromonos ---
[INFO] Loading execution data file /Users/lucas/Desktop/TQS/tqs-indiv-portfolio-lucasruivo/HW1/target/jacoco.exec
[INFO] Analyzed bundle 'zeromonos' with 9 classes
[INFO] BUILD SUCCESS
[INFO] Total time: 12.145 s
[INFO] Finished at: 2025-11-03T23:11:10Z
[INFO]
```

```
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.62 s -- in com.example.zeromonos.StaffFunctionalTest
[INFO] Results:
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.13:report (report) @ zeromonos ---
[INFO] Loading execution data file /Users/lucas/Desktop/TQS/tqs-indiv-portfolio-lucasruivo/HW1/target/jacoco.exec
[INFO] Analyzed bundle 'zeromonos' with 9 classes
[INFO] BUILD SUCCESS
[INFO] Total time: 12.147 s
[INFO] Finished at: 2025-11-03T23:12:08Z
[INFO]
```

3.5 Non-functional testing

Ficheiro: BookingPerformanceTest.java

Objetivo: Medir tempos de resposta básicos de endpoints (POST, GET por token, PUT estado, GET lista).

Implementação: Java 11+ HttpClient com timeouts, medição em ms (System.nanoTime) e asserts de 200 OK.

Tecnologias: Spring Boot Test (RANDOM_PORT), HttpClient, AssertJ.

```
2025-11-03T22:41:05.919Z INFO 27804 --- [o-auto-1-exec-1] c.e.z.boundary.BookingController : Listar todos os bookings
Tempo de listagem de todas reservas: 196ms
2025-11-03T22:41:06.108Z INFO 27804 --- [o-auto-1-exec-2] c.e.z.boundary.BookingController : Criar novo booking: município=Coimbra, data=2025-11-20, timeslot=11:00-13:00
Tempo de criação do bookings: 365ms
2025-11-03T22:41:06.444Z INFO 27804 --- [o-auto-1-exec-3] c.e.z.boundary.BookingController : Criar novo booking: município=Coimbra, data=2025-11-20, timeslot=15:00-17:00
2025-11-03T22:41:06.518Z INFO 27804 --- [o-auto-1-exec-4] c.e.z.boundary.BookingController : Atualizar booking token=bc131bd6-fc8e-42f1-alc1-ab650b7d5ec4 para estado=EM_PROG
Tempo de update do estado: 18ms
2025-11-03T22:41:06.537Z INFO 27804 --- [o-auto-1-exec-5] c.e.z.boundary.BookingController : Criar novo booking: município=Coimbra, data=2025-11-20, timeslot=13:00-15:00
2025-11-03T22:41:06.606Z INFO 27804 --- [o-auto-1-exec-6] c.e.z.boundary.BookingController : Consultar booking pelo token: e3a53e02-95e0-41d6-896f-db59098eb746
Tempo de fetch por token: 3ms
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.087 s - in com.example.zeromonos.BookingPerformanceTest
[INFO] Results:
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.13:report (report) @ zeromonos ---
[INFO] Loading execution data file /Users/lucas/Desktop/TQS/tqs-indiv-portfolio-lucasruivo/HW1/target/jacoco.exec
[INFO] Analyzed bundle 'zeromonos' with 9 classes
[INFO] BUILD SUCCESS
[INFO] Total time: 6.018 s
[INFO] Finished at: 2025-11-03T22:41:07Z
[INFO]
```

Nota: Os valores estão no print

3.6 Code quality analysis

zeromonos

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.example.zeromonos.service		93%		75%	9	33	0	48	1	13	0	2
com.example.zeromonos.data		97%		83%	4	31	1	51	0	19	0	3
com.example.zeromonos		37%		n/a	1	2	2	3	1	2	0	1
com.example.zeromonos.boundary		100%		100%	0	16	0	43	0	15	0	3
Total	25 of 663	96%	14 of 66	78%	14	82	3	145	2	49	0	9

Test Coverage com Jacoco

Main Branch Summary

702 Lines of Code • Last analysis 1 minute ago • 1e087c60

Quality Gate: Sonar way

✓

Passed

New Code

Overall Code

Security

5 Open Issues

B

Reliability

1 Open Issues

A

Maintainability

13 Open Issues

A

Accepted Issues

0

Coverage

A few extra steps are needed for SonarQube Cloud to analyze your code coverage.
[Set up coverage analysis](#)

Duplications

0.0%

No conditions set on 857 Lines

Security Hotspots

0

Code Quality with Sonar

A cobertura foi altíssima na maior parte do código, menos nos serviços cujo o número de branches perdidas era um pouco maior devido às variadíssimas opções da função `updateBookingStatus()`.

O Sonar foi muito útil, como por exemplo em casos de exceções genéricas em endpoints. O Sonar sinalizou uso de `RuntimeException` em fluxos 404 (reliability): foi trocado por `ResponseStatusException(404)`, melhorando semântica HTTP e manutenção.

4 References & resources

Project resources

Resource:	URL/location:
video.mov	Na raiz do projeto